FESI: A NOVEL MULTI-CORE CACHE COHERENCE PROTOCOL

A THESIS

Submitted by

RISHABH SRIKRISHNAN CHITOOR

for the award of the degree

of

BACHELOR OF TECHNOLOGY

DEPARTMENT OF ELECTRICAL ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI-600036

JUNE 2021

Dedicated to my sister...

THESIS CERTIFICATE

This is to certify that the thesis entitled **"FESI: A NOVEL MULTI-CORE CACHE COHERENCE PROTOCOL"** submitted by **RISHABH SRIKRISHNAN CHITOOR** to the Indian Institute of Technology, Madras for the award of the degree of **Bachelor of Technology** is a bona fide record of research work carried out by him under my supervision. The contents of this thesis, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Prof. Madhu Mutyam
Professor
Dept. of Computer Science and Engineering
Indian Institute of Technology Madras
Chennai – 600 036.

Prof. TG Venkatesh
Assistant Professor
Dept. of Electrical Engineering
Indian Institute of Technology Madras
Chennai – 600 036.

Place: Chennai

Date: June 2021

ACKNOWLEDGEMENTS

I would like to express my gratitude towards Prof. Madhu Mutyam and Prof. TG Venkatesh for their kind cooperation, guidance and encouragement which helped me in completion of this project.

In an academic year the whole of which I spent at home, I would like to thank my parents and sister for supporting me through the entire project. A special thanks to my father, Srikrishnan Chitoor for helping me with the intricate issues I faced while installing the required tools from source files, as was necessary on the AQUA cluster.

I would also like to thank Surya from the PACE Lab for helping me set up my AQUA cluster account.

ABSTRACT

Keywords: Cache Coherency ; Multi-core; Coherence Protocols; Computer Architecture.

Cache coherence protocols ensure data uniformity within the caches in a multi-core architecture. There are many cache coherent protocols currently such as MSI, MESI, etc. The most popular cache coherence protocols in modern day processors are the MESIF protocol (Intel) and the MOESI protocol (AMD). The significant delay in the performance of a cache system is when access to the Last-Level Cache (LLC) is required, since the lower level caches are significantly larger and slower than the Level 1 (L1) Caches. These LLC accesses can be split into two categories, LLC reads and LLC writes. LLC reads can be reduced by utilizing cache-to-cache transfers whenever a sibling core contains the required data block. LLC writes can be reduced by eliminating unnecessary write-backs. All the current protocols, specifically the most commonly used MOESI and MESIF do not achieve 100% utilization of cache-to-cache transfers over LLC reads, i.e., there exist scenarios when a sibling core has the data block but the block is supplied from the LLC. Both MESIF and MOESI also contain unnecessary write-backs.

In this thesis, we propose Forward-Exclusive-Shared-Invalid (FESI), a four state invalidation-based cache coherence protocol. FESI achieves the least number of accesses to the LLC compared to all other existing coherence protocols, thereby improving cache performance. This reduction in the number of LLC accesses is done by achieving a 100% utilization of cache-to-cache transfer of data blocks over supplying from the LLC and achieving minimal write-backs.

TABLE OF CONTENTS

ABSTRACT ii

TABLE OF CONTENTS iii

LIST OF FIGURES v

LIST OF TABLES vi

CHAPTER 1 7

INTRODUCTION 7

1.1. MULTI-CORE ARCHITECTURE AND COHERENCE PROTOCOLS 7

1.2. PERFORMANCE ANALYSIS OF COHERENCE PROTOCOLS 9

1.3. LIMITATIONS OF EXISTING PROTOCOLS 10

1.4. CONTRIBUTION OF THESIS WORK 11

CHAPTER 2 13

LITERATURE REVIEW 13

2.1. BACKGROUND 13

2.2. MSI PROTOCOL 13

2.3. MESI PROTOCOL 14

2.4. MOESI PROTOCOL 15

2.5. MESIF PROTOCOL 17

2.6. PROBLEM DEFINITION AND APPROACH 19

CHAPTER 3 20

THEORY AND IMPLEMENTATION OF FESI 20

3.1. ESSENCE OF FESI 20

3.2. COST VS BENEFIT OF ALLOCATING ‘F’ ON EVICTION 21

3.3. SNOOP-BASED FESI PROTOCOL 22

3.4. DIRECTORY-BASED FESI PROTOCOL: HIGH-LEVEL SPECIFICATION 23

3.5. DIRECTORY-BASED FESI PROTOCOL: DETAILED SPECIFICATION 26

CHAPTER 4 30

RESULTS AND DISCUSSION 30

4.1. SNOOP-BASED FESI PROTOCOL EXPERIMENTAL SETUP 30

4.2. SNOOP-BASED FESI PROTOCOL RESULTS 30

4.3. DIRECTORY-BASED FESI PROTOCOL EXPERIMENTAL SETUP 34

4.3.1 OVERVIEW OF GEM5 34
4.3.2 FESI IN GEM5 36
4.4. DIRECTORY-BASED FESI PROTOCOL RESULTS 37
CHAPTER 5 **39**
POSSIBLE ENHANCEMENTS TO FESI **39**
5.1. SENDING 'PUTF' WITHOUT DATA 39
5.2. ADDING THE 'M' STATE 40
CHAPTER 6 **42**
CONCLUSIONS AND FUTURE WORK **42**
REFERENCES **44**
APPENDIX: INSTALLING GEM5 FROM SOURCE FILES **45**

LIST OF FIGURES

Fig. 1.1 Schematic showing placement of Cache between the Processor and Main Memory 7

Fig. 1.2 Schematic showing which are private components and which are shared components in a multi-core architecture 8

Fig. 1.3 Schematic showing state transitions from I to S on a Processor Load request in the MSI protocol 10

Fig. 3.1 Schematic showing passing on responsibility of 'F' state to a Sharer during eviction 21

Fig. 3.2 High level state transition diagrams for FESI Directory-based implementation 25

Fig. 4.1 Bar graph showing comparison of number of writebacks across MSI, MESI, MESIF, MOESI and FESI 33

Fig. 4.2 Code showing example of Transitions in GEM5 Ruby SLICC files 35

Fig. 4.3 Code showing example of Actions in GEM5 Ruby SLICC files 36

Fig. 5.1 State transition diagram showing PutF+Data transition in FESI protocol 39

Fig. 5.2 State transition diagram showing 'PutF without data' enhancement to FESI protocol 40

Fig. A.1 Image showing list of pip packages required for GEM5 52

LIST OF TABLES

Table 2.1. Snoop-based MSI state transition table 14
Table 2.2. Snoop-based MESI state transition table 14
Table 2.3. Snoop-based MOESI state transition table 15
Table 2.4. Snoop-based MESIF state transition table 17
Table 3.1. Snoop-based FESI state transition table 22
Table 3.2. Cache controller for Directory-based FESI protocol 27
Table 3.3. Directory controller for Directory-based FESI protocol 28
Table 4.1. Results of memory trace 1, snoop-based FESI simulation 31
Table 4.2. Results of memory trace 2, snoop-based FESI simulation 31
Table 4.3. Results of memory trace 3, snoop-based FESI simulation 32
Table 4.4. Results of GEM5 simulation of Directory-based FESI protocol with 4 cores 37
Table 4.5. Results of GEM5 simulation of Directory-based FESI protocol with 12 cores 37
Table 5.1. GEM5 statistics to decide on sending PutF with data vs without data 39
Table 5.2. GEM5 statistics to show advantage in adding ‘M’ state 41

CHAPTER 1

INTRODUCTION

1.1. MULTI-CORE ARCHITECTURE AND COHERENCE PROTOCOLS

A computer primarily consists of three components; processor, memory and i/o. A lot of enhancements have been done to this basic model to substantially improve performance in modern-day computers. A few examples of enhancements to the processor include branch predictors, instruction level parallelism and multi-core architectures. In memory, the most significant advancement has been caches. As shown in **Fig. 1.1**, caches are smaller and faster memory placed between the main memory and the processor to substantially reduce the time taken to access data from memory.

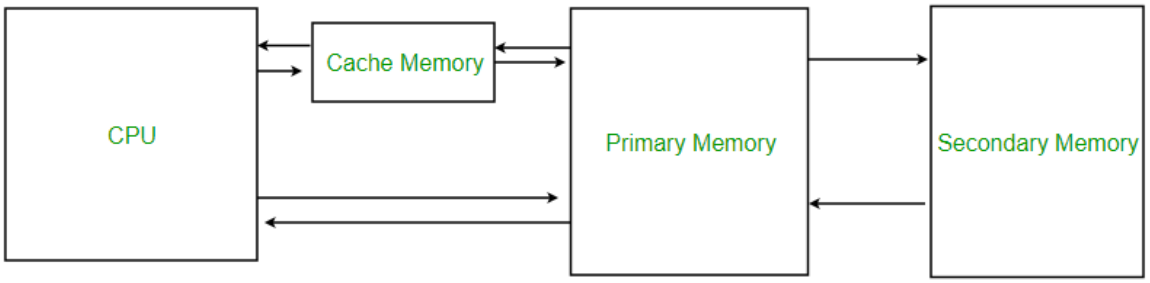


Fig. 1.1 Schematic showing placement of Cache between the Processor and Main Memory

The cache is faster, smaller and more expensive per bit of storage than the main memory. On receiving a Load/Store request for a memory location, the processor first searches the cache, and if it is a cache miss, the processor fetches the data from the main memory. A cache access is around 10-100 times faster than going to the main memory, thus a cache hit boosts the performance significantly. However, we cannot increase the size of caches beyond a point due to the high cost per bit . This requires us to devise efficient algorithms to store only those data blocks from the main memory that will be used frequently by the processor in the cache. Modern-day processors have multiple levels of caches, conventionally called L1 (Level 1), L2, etc., where L1 cache is the closest to the processor. The L1 cache is smaller and faster than the L2 cache, and so on. The L1 cache in modern-day processors is viewed as more of an extension of the processor than a part of the memory.

From the foundational single-core in-order processor, the processor has also evolved to give much better performance. In this project, we look into one such enhancement, multi-core

architectures. In multi-core architecture, there are multiple independent processors that share the same main memory. Each of these processors are called “cores”, and can run logical computations independently. A program is split into segments that can be run parallely, and these segments are run on different cores. Modern day laptops have between 4 to 8 cores.

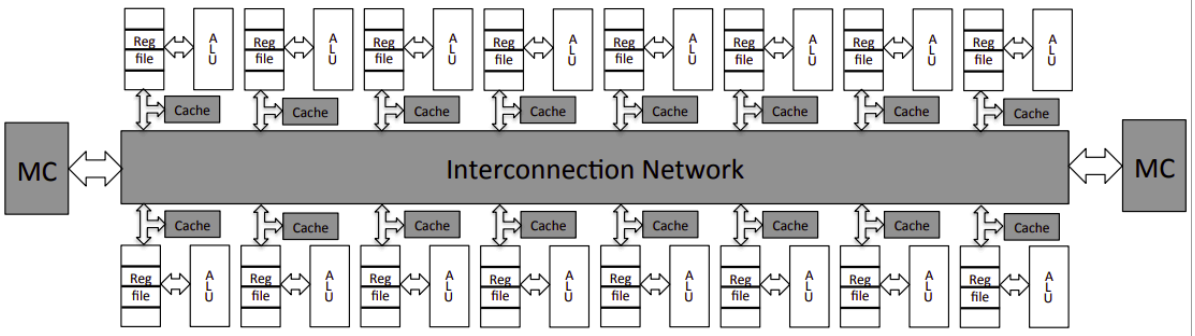


Fig. 1.2 Schematic showing which are private components and which are shared components in a multi-core architecture

Even though the cores can run logical computation parallely, the memory accesses are still sequential. To make maximum use of the multi-core architecture, we should enable the multiple cores to perform Load/Store operations simultaneously. This is achieved by cores having their private L1 caches, as shown in **Fig. 1.2**. The last-level cache (LLC) and main memory are still shared among the cores. This way, each core can run all computations independently, leading to better performance and lower power consumption in multi-core architectures compared to a single fast processor core. However, the presence of private L1 caches leads to the problem of *Cache Coherency*.

Cache coherency deals with the issue that arises when the L1 cache of a core does not contain the latest updated value of a data block. This would lead the core to read a stale value from its L1 cache thus causing wrong results. Imagine an architecture with two cores C1 and C2. The following load/store requests are processed, where ‘a’ has value of ‘5’ initially:

- C1 - Load &a: Fetch from LLC and block is stored in C1 L1 cache
- C2 - Load &a: Fetch from LLC and block is stored in C2 L1 cache
- C1 - Store &a,10: L1 cache hit, value of data block updated in the C1 L1 cache to 10
- C2 - Load &a: L1 cache hit, read value of 5 instead of 10

The issue of cache coherency arises in both write-through and write-back caches. If the above example implemented write-through caches, and updated the LLC on the store operation, core C2 would still read the stale value since there is a L1 Cache hit which holds the stale value.

Thus, we need the L1 caches to be coherent at any point of execution. Cache coherence is enforced using cache coherence protocols. Cache coherence protocols can be broadly split into the following two types:

- Invalidation-based: Invalidates the cache copies in all other cores before writing.
- Update-based: Updates the cache copies in all other cores with latest data on each write.

While the update-based method looks better and faster, repeated writes to the same block would need repeated updates but only one invalidate. Thus, most protocols are invalidation-based. Additionally, we can classify cache coherence protocols based on the network used:

- Snoop / Bus based: Send requests through the bus to other cores/LLC.
- Directory based: All requests go through the directory, more scalable.

The directory network is a point-to-point communication where on receiving a request from a core, the directory either provides the data block or reroutes the request to the specific core that can provide the data block. On the other hand, in a bus-based network, the request is broadcasted to all the cores, making it less efficient. However, the bus based method occupies less space since separate directory entries need not be maintained.

1.2. PERFORMANCE ANALYSIS OF COHERENCE PROTOCOLS

Performance of any cache system is measured by the Average Memory Access Time (AMAT). The AMAT is given by:

L1 Hit Latency + (1 - L1 Hit Rate) * L1 Miss Latency

The L1 cache hit rate and hit latency are not dependent on the cache coherence protocol. The cache coherence protocol affects only the L1 miss latency. Thus, while designing cache coherence protocols, in addition to ensuring coherence, the objective is to reduce the L1 cache miss latency. The following observations help in achieving this objective:

- Cache-to-cache transfer of data blocks is faster than accessing the data block from the LLC/main memory.
- Writebacks consume significant time since an access to LLC/main memory is required.

Thus, to improve the performance of a cache coherence protocol, we need to increase the chance of a sibling core being able to provide the data block, and reduce the need for write-backs to LLC.

1.3. LIMITATIONS OF EXISTING PROTOCOLS

On analyzing the current cache coherence protocols, we realized that each of these protocols falter in both of the two above-mentioned parameters. In each of these protocols, there are either instances where a data block is fetched from the LLC when it could have been provided by a sibling core, or a write-back is issued unnecessarily. As an example, let us look at the MSI protocol.

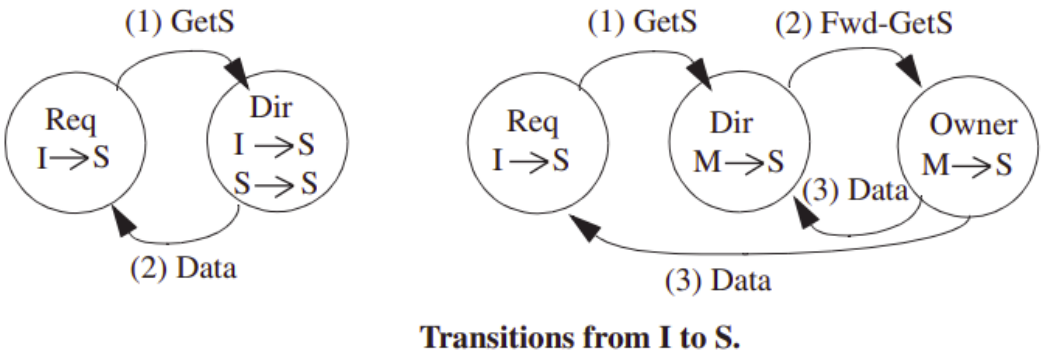


Fig. 1.3 Schematic showing state transitions from I to S on a Processor Load request in the MSI protocol

Fig. 1.3 is a part of the state transition diagrams for the directory implementation of the MSI protocol given in Sorin et al. [3].

From the state diagram on the left, we can see that when multiple cores have the data block in 'S' state (Directory in 'S' state), and a request for the data block arrives, the LLC supplies the block even though one of the other cores could have supplied it. This is because the protocol cannot choose which among the many sharers should supply the data block.

From the state diagram on the right, we can see that when a core has the required data block in the 'M' state (Owner of the block), the directory forwards the request to this core, which

then supplies the data block to the requestor. Once the owner supplies the block to the requestor, both cores have the block in 'S' state. However, there is an "unnecessary" write-back to the LLC in this process. This write-back is required because both the owner and requestor move to 'S' state after this process, which means the next time a request for the block arrives, the LLC has to supply the data block. Since the LLC has to supply the latest copy of the data block, the "Modified" data block needs to be written back.

In this way, the MSI protocol contains unnecessary reads to the LLC which subsequently causes unnecessary writes to the LLC. More such drawbacks of the other existing protocols including MESIF, MOESI, etc. are elaborated in Chapter 2 of this thesis.

1.4. CONTRIBUTION OF THESIS WORK

An ideal cache coherence protocol should obtain a data block from the LLC only when no other sibling core contains the data block in its L1 cache. In other words, if even one sibling core has the data block in its L1 cache, the data block should not be fetched from the LLC. Additionally, an ideal cache coherence protocol should write-back to the LLC only when absolutely necessary. The second goal is more easily achieved once the first goal is achieved.

In this thesis, we propose Forward-Exclusive-Shared-Invalid (FESI), a four state invalidation-based cache coherence protocol. In this project, we implement "FESI" using both directory and bus networks. More details of the experimental analysis and results will be provided in Chapter 4 of this thesis.

The "FESI" protocol is designed such that if any sibling core contains the requested data block, it will be provided through cache-to-cache transfer and the LLC will not be accessed. In a sense, there is 100% utilization of cache-to-cache transfer, which is not achieved by the existing protocols. This automatically leads to the elimination of all unnecessary write-backs. As a result, for any program, the number of LLC accesses in "FESI" protocol is less than or equal to the number of LLC accesses in any other protocol. More details about the theory and implementation of the FESI protocol are provided in Chapter 3 of this thesis.

The thesis organization is as follows: We review the existing protocols and elaborate on their drawbacks in chapter 2. We then talk about the contribution of this thesis, the theory and implementation of FESI in chapter 3. In chapter 4, we provide the results and experimental analysis of the FESI protocol. Based on the results obtained, we discuss possible alternate implementations and enhancements to the FESI protocol in chapter 5. We give the conclusion

and future work in chapter 6. Additionally, we have attached an appendix which details how to build the GEM5 simulator entirely from source files.

CHAPTER 2

LITERATURE REVIEW

2.1. BACKGROUND

In 2001, IBM introduced the world's first multi-core processor. Cache coherence is one of the most integral requirements for a multi-core processor, and as a result, cache coherence is one of the most looked-into problems in computer architecture. Due to this, numerous cache coherence protocols have been proposed. In this chapter, we focus on the 4 most popular existing protocols; MSI, MESI, MOESI and MESIF. We provide a brief explanation about working of each of these protocols in addition to elucidating their drawbacks.

We use the following abbreviations while describing the existing cache coherence protocols in the subsequent sections of this chapter:

- ProcRd: A load request from the processor to the data block.
- ProcWr: A store request from the processor to the data block.
- BsRd: A request from another core to read from the data block.
- BsRdX: A request from another core to write into the data block.
- Eviction: The block is evicted from cache due to the replacement policy.

For the drawbacks,we exclusively focus on depicting scenarios where the protocol will:

- Supply requested block from LLC when one or more sibling cores have the block.
- Perform unnecessary write-backs.

2.2. MSI PROTOCOL

The MSI protocol contains the following 3 states:

- Modified: Core has the only copy of the block; Dirty; Read/Write privileges.
- Shared: Other cores have the block; Clean; Read Only.
- Invalid: Core does not contain the data block.

Table 2.1. Snoop-based MSI state transition table

State	ProcRd	ProcWr	BsRd	BsRdX	Eviction
M	Hit	Hit	supply, writeback, move to S	supply, move to I	writeback, move to I
S	Hit	Send BsRdX, move to M	-	move to I	move to I
I	send BsRd, move to S	Send BsRdX, move to M	-	-	-

Drawbacks:

- When sibling cores have the block in 'S' state, for an incoming BsRd request, the block is supplied from the LLC instead of cache-to-cache transfer.
- When core has the block in 'M' state, it needs to do a write-back to LLC while processing a BsRd request. This is because the LLC needs to have the latest updated value of the block so that it can supply for the next BsRd request.

2.3. MESI PROTOCOL

The MESI protocol contains the following 4 states:

- Modified: Core has the only copy of the block; Dirty; Read/Write privileges.
- Exclusive: Core has the only copy of the block; Clean; Read/Write privileges.
- Shared: Other cores have the block; Clean; Read Only.
- Invalid: Core does not contain the data block.

Table 2.2. Snoop-based MESI state transition table

State	ProcRd	ProcWr	BsRd	BsRdX	Eviction
M	Hit	Hit	supply, writeback, move to S	supply, move to I	writeback, move to I
E	Hit	Hit, move to M	supply, move to S	supply, move to I	move to I

S	Hit	Send BsRdX, move to M	-	move to I	move to I
I	send BsRd, move to E or S after check with other cores	Send BsRdX, move to M	-	-	-

The MESI protocol has the extra 'E' state which enables silent transition from E to M state on a ProcWr. This saves a bus transaction. However, extra hardware is required to determine whether a core goes to 'E' or 'S' state on a ProcRd from 'I' state.

Drawbacks:

- When sibling cores have the block in 'S' state, for an incoming BsRd request, the block is supplied from the LLC instead of cache-to-cache transfer.
- When a core transitions from 'M' to 'S' state while processing a BsRd request, it needs to do a write-back to LLC. This is because the LLC needs to have the latest updated value of the block so that it can supply for the next BsRd request.

The drawbacks are the same as the MSI protocol. This is because even now, when there are multiple sharers, MESI cannot determine which sharer will supply the block.

2.4. MOESI PROTOCOL (AMD)

The MOESI protocol contains the following 5 states:

- Modified: Core has the only copy of the block; Dirty; Read/Write privileges.
- Owned: Core is the Owner of block among Sharers; Dirty; Read Only.
- Exclusive: Core has the only copy of the block; Clean; Read/Write privileges.
- Shared: Other cores have the block; Clean; Read Only.
- Invalid: Core does not contain the data block.

Table 2.3. Snoop-based MOESI state transition table

State	ProcRd	ProcWr	BsRd	BsRdX	Eviction
M	Hit	Hit	supply, move to O	supply, move to I	writeback, move to I

O	Hit	send BsRdX, move to M	supply block	supply, writeback, move to I	write back, move to I
E	Hit	Hit, move to M	supply, move to S	supply, move to I	move to I
S	Hit	Send BsRdX, move to M	-	move to I	move to I
I	send BsRd, move to E or S after check with other cores	Send BsRdX, move to M	-	-	-

The MOESI protocol designates an "Owner" state so that when multiple sharer cores are present, the protocol can choose which sharer will supply the block. This way, in the scenario of multiple sharers, an incoming BsRd request will be processed using cache-to-cache transfer instead of having to go to the LLC. This is a significant enhancement from MSI and MESI since we have converted an LLC access to a cache-to-cache transfer. This also leads to the removal of unnecessary write-back to LLC during transition from "M" to "S", or in this protocol, "M" to "O".

Drawbacks:

- The 'O' state is reachable only on a BsRd from 'M' state. If the multiple cores only process read requests, 'M' state will never be reached, and subsequently 'O' state will never be reached. Essentially, a ProcWr is necessary to reach the 'O' state. Thus, in scenarios where there is no ProcWr for a significant amount of time, there will be multiple sharer cores with none of them being the owner. Thus the LLC will have to supply the data block on a BsRd request even when sharer cores are available.
- Once a block becomes an owner, it doesn't pass on the responsibility to other sharers, so there is a significant workload burden on a single core. This will affect the owner's single core performance.
- The owner does not reflect the usage patterns of the data block in the overall system. If the owner core does not access the specific data block frequently while the other sharer cores are regularly accessing the block, the owner would still get evicted even though the block is in high demand in the overall system. Thus, if the owner gets

evicted but there is significant demand in the system for the block, the LLC will have to supply even though multiple sharer cores are present.

- At the same time, we cannot protect the 'O' state from eviction or keep setting it to MRU since it would be unfair to that specific core's usage patterns.
- A BusRdX request to the 'O' state requires a write-back, which is unnecessary.

While the MOESI protocol is a significant improvement compared to MSI and MESI protocols in increasing the chances of core-to-core transfers, there are still instances when sharer cores are present but the LLC is accessed for the data block.

2.5. MESIF PROTOCOL (Intel)

The MESIF protocol contains the following 5 states:

- Modified: Core has the only copy of the block; Dirty; Read/Write privileges.
- Exclusive: Core has the only copy of the block; Clean; Read/Write privileges.
- Shared: Other cores have the block; Clean; Read Only.
- Invalid: Core does not contain the data block.
- Forward: Special state among sharers to forward the requested data; Clean; Read Only.

Table 2.4. Snoop-based MESIF state transition table

State	ProcRd	ProcWr	BsRd	BsRdX	Eviction
M	Hit	Hit	supply, writeback, move to S	supply, writeback, move to I	writeback, move to I
E	Hit	Hit, move to M	supply, move to S	supply, move to I	move to I
S	Hit	Send BsRdX, move to M	-	move to I	move to I
I	send BsRd, move to E or F after check with other cores	Send BsRdX, move to M	-	-	-
F	Hit	Send BsRdX, move to M	supply, move to S	supply, move to I	move to I

The MESIF protocol designates one of the sharer cores to ‘F’ state, and it will be this core which supplies the data block on BsRd or BsRdX requests. This responsibility of the ‘F’ state is passed among the sharers based on who requested for the data block most recently. The two important differences between this ‘F’ state and ‘O’ state of MOESI is that ‘F’ is clean while ‘O’ is dirty, and the ‘F’ keeps moving among the sharers while the ‘O’ state is held by a single core till invalidated or evicted.

Since the ‘F’ state keeps moving among the sharers, it reflects the usage patterns of the overall system. The core that most recently requested for the block (and thus lowest chance of eviction) will hold the ‘F’ state. Hence, it is highly unlikely that a block in the ‘F’ state gets evicted. This further increases the chances of cache-to-cache transfers compared to LLC accesses.

Drawbacks:

- Since ‘F’ is a clean state, the ‘M’ to ‘S’ transition on a BsRd requires a write-back to the LLC. The ‘O’ state did not require this write-back.
- The “Cache Hits” on ProcRd for other shared cores do not decide which core holds the ‘F’ state. It only counts which of the cores last transitioned from ‘I’ to ‘F’. For example, consider a 4 core system. Core 1 and core 2 frequently access the data block. After a while, core 3 sends a BsRd request for the data block. Upon receiving the data block from either core 1 or core 2, it transitions to ‘F’ state and never accesses the data block again. Meanwhile core 1 and core 2 continue to regularly access the data block. Soon, the data block would get evicted from core 3, which means the next time either core 3 or 4 request a BsRd or BsRdX for the data block, the LLC would need to be accessed.

The MESIF and MOESI protocols solve each others’ problems in a way. MESIF reflects the usage patterns of the overall system better than MOESI. It also distributes the responsibility of supplying the block among the sharers, and thus no single core is burdened with a high workload. However, MOESI does not need a write-back during the ‘M’ to ‘S’ transition on BsRd.

The problem still remains that in case of eviction of ‘F’ block, an LLC access would be needed for BsRd or BsRdX requests even when sharer cores are available.

2.6. PROBLEM DEFINITION AND APPROACH

From the above discussion, we can see that the MESIF and MOESI protocols are the most efficient of the existing protocols when it comes to utilizing cache-to-cache transfers over LLC accesses. This is also why these two are the most popular protocols today. However, there are still instances when both of these protocols access the LLC when sharer cores are available. Additionally, the MESIF protocol requires a write-back during 'M' to 'S' transition. The problem definition is to design a coherence protocol that achieves 100% efficiency in utilizing cache-to-cache transfers over LLC accesses, and requires the least number of write-backs to the LLC.

In the quest for this new coherence protocol, we can learn from the advantages of both the MOESI and MESIF protocols.

- MOESI enables dirty sharing of cache blocks. This helps eliminate the write-back during 'M' to 'S' transition.
- MESIF keeps rotating the responsibility of 'F' state among the sharers on the basis of "Most Recently Used". This results in less chance of the "supplier" core evicting the data block. This also ensures no single core bears a high workload.

The "FESI" protocol combines the above two features, one each from MESIF and MOESI, in addition to other innovative additions to solve the problem statement.

CHAPTER 3

THEORY AND IMPLEMENTATION OF "FESI"

3.1. ESSENCE OF "FESI"

"FESI" enables dirty sharing of data blocks, as well as rotating the responsibility of the 'F' state among the sharers based on "Most Recently Used". Basically, the 'F' in FESI is the same as 'F' in MESIF except now 'F' is dirty.

The FESI protocol contains the following 4 states:

- Exclusive: Core has the only copy of the block; Clean; Read/Write privileges.
- Shared: Other cores have the block; Dirty; Read Only.
- Invalid: Core does not contain the data block.
- Forward: Special state among sharers to forward the requested data; Dirty; Read Only.

Thus, we combine the best of MOESI and MESIF to ensure that:

- A write-back during 'M' to 'S' transition is not required as 'F' is a dirty state.
- Responsibility of supplying is rotated among sharers so the single core performance of the supplier (owner) is not degraded.
- This rotation of 'F' state is done on "Most Recently Used" basis that reduces the chance of eviction of the supplier.

Even now, the goal of 100% efficiency in utilization of cache-to-cache transfers over LLC accesses is not achieved during an eviction of the 'F' state. It would mean there are other sharer cores but LLC access is required for the next BsRd/BsRdX request. Additionally, since the 'F' state is dirty, each eviction of 'F' state requires write-back to cache which is unnecessary. To achieve this 100% efficiency, we propose the following enhancement in the FESI protocol:

When a core evicts a block in the 'F' state, the 'F' state is passed on to one of the other sharers if sharers exist. Only if no sharer core is present does it write-back to the LLC. This allocation of 'F' state to one of the other sharer cores is based on which sharer core responds fastest to the request, thereby boosting performance.

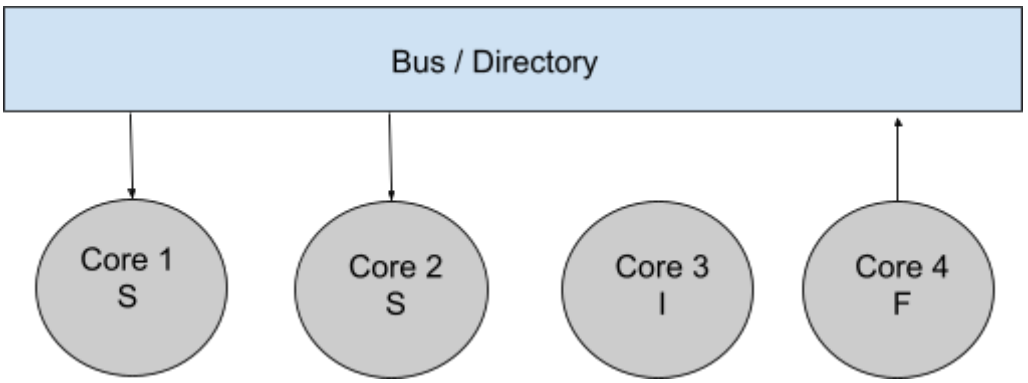


Fig. 3.1 Schematic showing passing on responsibility of 'F' state to a Sharer during eviction

For example, take **Fig. 3.1**, where core 4 has the block in 'F' state and is evicting it. It sends a request to all other Sharer Cores. Depending on which of core 1 and core 2 respond first, that is allocated as the new 'F' state and core 4 moves to 'I'. In this case, write-back is not required. If there are no sharers, then core 4 writes back to LLC and moves to 'I'.

Assume core 2 gets allocated as 'F' through this process. It might be that the block is next in line to get evicted in Core 2 due to the Cache replacement policy. That would mean we need to repeat the whole process again. To avoid this, when a core gets allocated as the 'F' state through this process, it is set to MRU in that core's L1 cache to boost the chances of the block not getting evicted.

After implementation of the above, we achieve 100% efficiency in utilizing cache-to-cache transfers over LLC accesses. If there are sharer cores with the requested data block, there will be at least one among the sharer cores with the block in 'F' or 'E' state. Thus, the data block will be provided through cache-to-cache transfer and LLC access is eliminated. Additionally, since we keep passing on the 'F' state on eviction, we defer the write-backs thus reducing unnecessary write-backs.

3.2.COST VS BENEFIT OF ALLOCATING 'F' ON EVICTION

The cost in passing on 'F' block on eviction is the additional bus transactions. On each eviction, we need to send a request to each sharer. Since we allocate the fastest responder as 'F', there is little impact on performance when other sharers are present. However, when there is no sharer present, we need to wait for acknowledgement of the same from each core before proceeding to write-back in a bus-based network. This might impact the overall performance. This problem is not present in a directory-based network since the directory

controller knows immediately how many sharers are present for the data block, and whether there are no sharers.

Further, the likelihood of a sharer being available only increases with an increase in the number of cores. Thus, the impact on performance while waiting for acknowledgements when no sharer is present decreases with increasing number of cores. Additionally, the limited overheads of this process are further mitigated by setting the newly allocated 'F' block as MRU to avoid repeating the process frequently.

The benefits of passing on 'F' during eviction are many:

- Write-back on eviction of dirty 'F' state is deferred if another sharer core is present. Essentially, for every single data block, only a single write-back is required, when the final sharer core is evicted. This is unlike MOESI where every eviction of 'O' required write-back.
- The next BsRd/BsRdX request can be processed through cache-to-cache transfer instead of LLC access.

3.3.SNOOP-BASED FESI PROTOCOL

Table 3.1. Snoop-based FESI state transition table

State	ProcRd	ProcWr	BsRd	BsRdX	Eviction
E	Hit	Hit, move to F	supply, move to S	supply, move to I	move to I
S	Hit	Send BsRdX, move to F	-	move to I	move to I
I	send BsRd, move to E or F after check with other cores	Send BsRdX, move to F	-	-	-
F	Hit	Hit, Send BsRdX	supply, move to S	supply, move to I	pass or write-back, move to I

Similar to all protocols with the 'E' state, we need extra hardware in the snooping protocol to determine whether to move 'I' to 'F' or 'I' to 'E' on a ProcRd request. FESI uses this same hardware to determine if there are other sharers for the data block during eviction of 'F'.

One point to note is that there is no 'M' state. This means there is no state with Read/Write privileges. On a ProcWr to the 'F' state, we need to repeatedly send BsRdX requests on the bus even if there are no other sharers. During consecutive repeated writes to the same data block, even though the first BsRdX would invalidate all sharers, and the subsequent writes would have no sharers, we still need to send BsRdX requests to all cores on ProcWr to 'F' state leading to unnecessary bus transactions.

A simple fix to this would be to add the 'M' state and make it a five state protocol like MESIF and MOESI. However, this would result in increases in storage since we need only 2-bits to denote 4 states while we need 3-bits to denote 5 states. This is a trade-off between better performance and lower memory occupied, and is not the focus of this thesis.

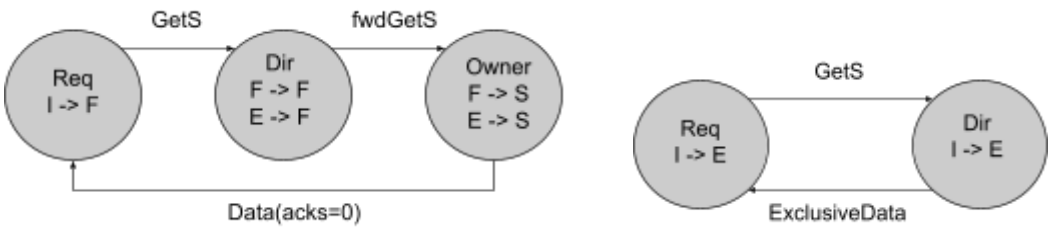
3.4. DIRECTORY-BASED FESI PROTOCOL: HIGH-LEVEL SPECIFICATION

The directory-based FESI protocol has an advantage over the snoop-based protocol since the directory controller knows exactly the sharers and number of sharers for a data block. Thus, it enhances four features from the snoop-based FESI protocol:

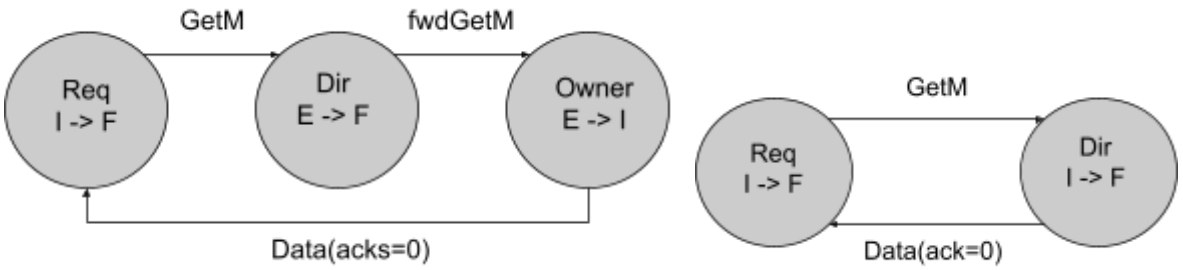
- On a 'F' eviction, the request for passing on 'F' state is sent only to the sharer cores instead of all cores.
- On a 'F' eviction, the directory would know immediately if there are no other sharers. Thus, we don't need to wait for an acknowledgement from each core before writing back to LLC.
- On repeated writing to a 'F' block, the BsRdX need not be sent repeatedly to all the cores. This is because the directory controller knows the number of sharers is 0 for all BsRdX requests after the first BsRdX. Thus, the issue of lack of 'M' state is ameliorated.
- Extra hardware for determining whether to move 'I' to 'F' or 'I' to 'E' on a ProcRd request is not required since the directory knows the number of sharers.

The state transition diagrams for the directory implementation of FESI are shown in **Fig. 3.2**. The notation followed in the state transition diagrams are:

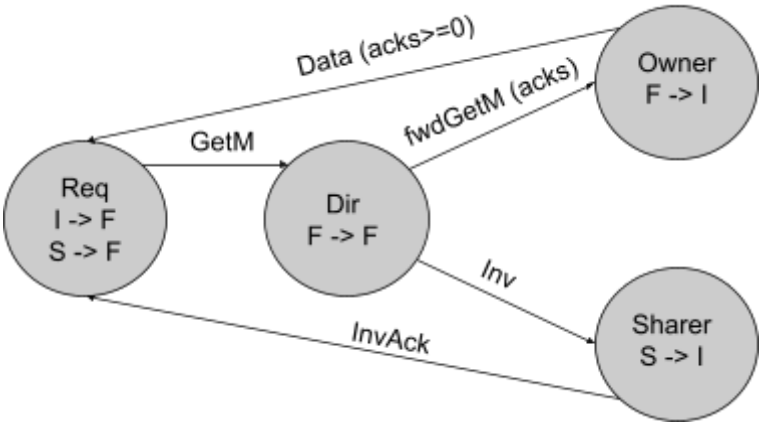
- Req: Requestor, the core requesting the data block.
- Dir: Directory Controller.
- Owner: The designated supplier of the block, which is either in ‘E’ or ‘F’ state.
- Sharers: All the cores having the block in ‘S’ state. Note; the core with the block in ‘F’ or ‘E’ state is not counted among the sharers.
- GetS: Similar to BsRd request.
- GetM: Similar to BsRdX request.
- Inv: Invalidate the data block (due to GetM request to the block).
- Acks / AckCount: Number of invalidation acknowledgements core should wait for before proceeding. This is equal to the number of sharer cores for the data block.

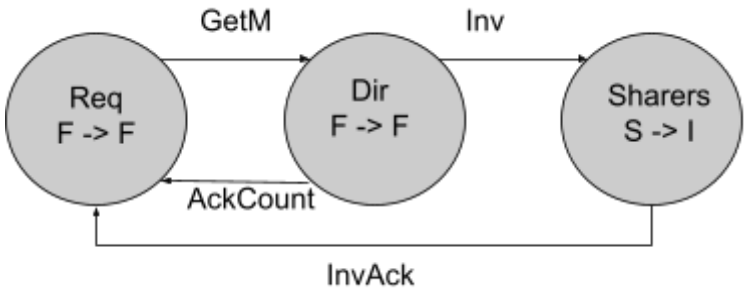


Transitions from I on a ProcRd (to E or F)

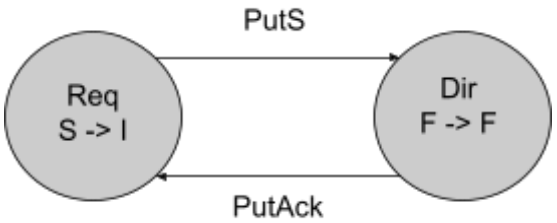


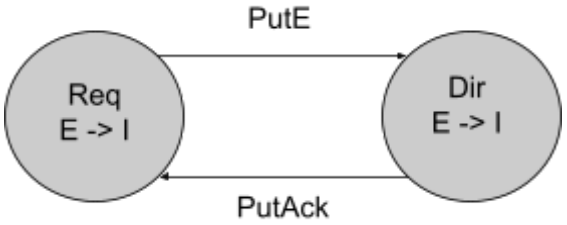
Transitions from I to F on a ProcWr (0 Sharers)

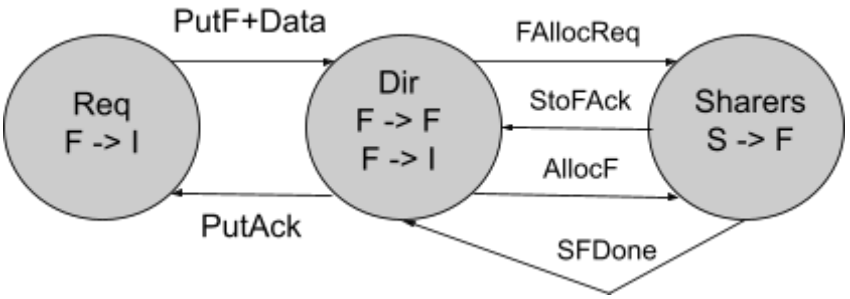




Transition to F on ProcWr (>0 Sharers)







Transition during Evictions of E, F, S blocks

Fig. 3.2 High level state transition diagrams for FESI directory-based implementation

From the above state transition diagrams, we elaborate on a few of the transitions:

- The 'PutF' request (on eviction of F) is always sent with data. If there is another sharer core to which directory can pass on the 'F' state, this data is not used. If not, then this data is written back to the LLC.
- On a PutF request, the directory sends "FAllocReq" to all the sharers. "AllocF" is sent to only one among the sharers, the one which responds first with "StoFAck".

- The sharers on receiving “FAllocReq” can either respond with “StoFAck” or “StoFInvAck” to specify whether they can or cannot accept the ‘F’ state. The chosen sharer on receiving “AllocF” will move from ‘S’ to ‘F’ and acknowledge with “SFDone”.
- On a ‘F’ to ‘F’ transition during ProcWr/GetM, the core already has the data. Thus, the directory responds only with the “AckCount” and does not send data again.
- “ExclusiveData” is a separate “MessageType” sent by directory to inform the core to move to ‘E’ instead of ‘F’ (from ‘I’ on a ProcRd).

To avoid deadlocks during the state transitions, we classify messages into three different virtual networks in the FESI protocol:

- Request: All the requests from requestor to directory. Ex. GetS, GetM, PutE.
- Response: All responses, either from directory to requestor, or from owner/sharers to the requestor. Ex. AckCount, InvAck.
- Forward: All forwarded requests from the directory to the owner/sharers. Ex. fwdGetS, fwdGetM, Inv, PutAck*.

By utilizing these three networks, we can avoid a response getting stuck behind a request, and so on. This also ensures avoidance of livelocks.

*NOTE: For the same purpose of avoiding deadlocks, PutAcks are sent through the forward network instead of the response network.

3.5. DIRECTORY-BASED FESI PROTOCOL: DETAILED SPECIFICATION

In the tables below, the detailed specification of the directory-based FESI protocol is given. This includes all the transition states and the actions taken during each transition. There are two tables, one for the L1 cache controller and the second for the directory controller. The transient states in the tables are for capturing transitions from one state to another, where they are either waiting for data or some acknowledgement.

For example, on transition from ‘I’ to ‘E’ or ‘F’ on a ProcRd request, the core needs to wait for the data block after sending the GetS request. This intermediary state after sending the GetS is represented by IEd in the table below. Then, if IEd state gets “ExclusiveData from Dir”, we move on to ‘E’ thus completing the ‘I’ to ‘E’ transition. Else, if IEd state gets “Data from owner”, we move on to ‘F’ thus completing the ‘I’ to ‘F’ transition.

Table 3.2. Cache controller for Directory-based FESI protocol

	Load	Store	Replace	Fwd GetS	Fwd GetM	Inv	PutAck	Exclusive Data from Dir	Data from Dir	Data Owner (ack>0)	Data Owner (ack = 0)	InvAck	Last InvAck	AckCount from Dir (>0)	AckCount from Dir (=0)	FAllocReq	AllocF
I	send GetS / IEd	send GetM / IFad	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
E	hit	hit / F	PutE / EIa	send data to req / S	send data to req / I	-	-	-	-	-	-	-	-	-	-	-	-
F	hit	send GetM / FFaa	PutF + data / FIa	send data to req / S	send data to req / I	-	-	-	-	-	-	-	-	-	-	-	-
S	hit	send GetM / SFad	PutS / SIa	-	-	sendInvAck / i	-	-	-	-	-	-	-	-	-	send StoFAck + MRU	send SFDone / F
IEd	stall	stall	stall	stall	stall	-	-	- / E	-	-	- / F	-	-	-	-	-	-
IFad	stall	stall	stall	stall	stall	-	-	-	- / F	add Acks / IFa	- / F	ack--	-	-	-	-	-
IFa	stall	stall	stall	stall	stall	-	-	-	-	-	-	ack--	- / F	-	-	-	-
SFad	stall	stall	stall	stall	stall	sendInvAck / IFad	-	-	-	add Acks / SFa	- / F	ack--	-	-	-	send StoFAck + MRU	send SFdone / FFaa
SFa	stall	stall	stall	stall	stall	-	-	-	-	-	-	ack--	- / F	-	-	-	-
FFaa	stall	stall	stall	send data to req / SFad	send data to req / IFad	-	-	-	-	-	-	ack--	-	addReqdAcks to TBE / FFa	- / F	-	-
FFa	stall	stall	stall	stall	stall	-	-	-	-	-	-	ack--	- / F	-	-	-	-
EIa	stall	stall	stall	send data to req / SIa	send data to req / IIa	-	- / I	-	-	-	-	-	-	-	-	-	-
FIa	stall	stall	stall	send data to req / SIa	send data to req / IIa	-	- / I	-	-	-	-	-	-	-	-	-	-
SIa	stall	stall	stall	-	-	sendInvAck / IIa	- / I	-	-	-	-	-	-	-	-	send StoFInvAck	-
IIa	stall	stall	stall	-	-	-	- / I	-	-	-	-	-	-	-	-	-	-

Table 3.3. Directory controller for Directory-based FESI protocol

	GetS	GetMNonOwner	GetM Owner	PutS_ Sharer	PutS_ NotSharer	PutF+ data from Owner (Shares>0)	PutF+ data from Owner (Shares=0)	PutF+ data from NonOwner	PutE from Owner	PutE from NonOwner	StoFAck	StoFInvAck	LastStoFInvAck	SFdone
I	send ExclusiveData to req + Set req as Owner / E	send Data to req + Set req as Owner / F	-	send PutAck to req	send PutAck to req	-	-	send PutAck to req	-	send PutAck to req	-	-	-	-
E	send FwdGetS + Add Owner to sharer + Clear Owner + Set req as Owner / F	send FwdGetM to Owner + Clear Owner + Set req as Owner / F	send AckCount=0 to req	send PutAck to req	send PutAck to req	-	Write data to Mem + Clear Owner + send PutAck to req / I	send PutAck to req	Clear Owner + send PutAck to req / I	send PutAck to req	-	-	-	-
F	send FwdGetS to Owner + Add Owner to sharer's list + Clear Owner + Set req as Owner	remove Req from sharer + send FwdGetM + Clear Owner +Send Inv to sharers + Clear sharers + Set req as Owner	send AckCount to req + send Inv to sharers + Clear sharers	remove req from Sharers + send PutAck to req	send PutAck to req	clear Owner + send FAlloc Req to all sharers / Fas	Write data to Mem + Clear Owner + send PutAck to req / I	remove req from sharers + send PutAck to req	-	remove req from sharers + send PutAck to req	Pop Queue	Pop Queue	Pop Queue	-
Fas	-	-	-	-	-	stall	stall	stall	-	-	set responder as Owner + Remove Responder from sharers + send AllocF to responder / FFa	remove Responder from sharers	clear sharers + Write data to Mem + sendPutAck to Req / I	-
FFa	-	-	-	-	-	stall	stall	stall	-	-	Pop Queue	Pop Queue	Pop Queue	sendPutAck / F

The columns in the above tables are the messages which are incoming to the cache or directory controller. All the “responses” and “forwarded requests” that a core can receive (refer to **Fig. 3.2**) form the columns of the cache controller table. Similarly, all the “requests” and “responses” that a directory can receive (refer to **Fig. 3.2**) form the columns of the directory controller table.

Further, some of the possible messages are split into two separate events to help in easier implementation. For example, as seen in **Fig. 3.2**, the sharers send “InvAcks" to the requestor core on receiving an “Inv” request from the directory. The requestor moves on only when it has received the required amount of “InvAcks”. For easier implementation, this “InvAck” message is split into “InvAck” and “LastInvAck” columns in the cache controller table.

Similarly, in the directory controller table, the “GetM” request is split into “GetM owner” for GetM requests from cores in ‘F’ state and “GetM non-owner” for GetM requests from cores in ‘I’ and ‘S’ state to ease implementation. This way, for “GetM owner”, directory needs to respond only with the number of ‘acks’ while for “GetM non-owner”, directory needs to respond with both the data and the number of ‘acks’.

On first glance, the tables might be intimidating, but they give the most detailed specification of the protocol. The corresponding tables for the directory implementations of MSI, MESI and MOESI protocols can be found in the section 8.2 of Sorin et al. [3].

CHAPTER 4
RESULTS AND DISCUSSION

4.1. SNOOP-BASED FESI PROTOCOL EXPERIMENTAL SETUP

We build a snoop-based cache coherence simulator that takes memory traces and the required protocol to be simulated as input, and runs the memory trace with the chosen protocol. The input memory trace is of the form "read/write core_Number <address>". The simulator gives various statistics as output, such as number of loads, load misses, stores, write-backs, invalidations, etc. The cache coherence simulator has been extensively verified using numerous custom traces and outputs validated with other existing simulators. The cache coherence simulator supports the snoop-based MSI, MESI, MOESI, MESIF and my novel FESI protocols.

The cache coherence simulator is entirely built using C++ in an object oriented fashion.

The results given in the next section are for the cache coherence simulator run with 16 cores, cache set associativity of 4 and number of sets equal to 4. Such a small Cache is considered because we run memory traces with an average of only 100,000 Loads and Stores.

The code for the originally built cache coherence simulator can be found in this link: https://github.com/rishabh-c-s/FESI_protocol/tree/master/CacheSimulatorSnoopBased

NOTE: The cache coherence simulator is a bare-metal simulator meant to only measure the basic cache statistics on running a memory trace. It is meant to be an preliminary indicator of the effectiveness of the FESI protocol.

- This simulator does not measure the time taken for simulation.
- It does not do any full system simulation, nor can it run entire programs. It only takes memory traces in a specific format and simulates the L1 caches.

A detailed analysis is done in **Section 4.3** where we simulate FESI using the GEM5 Ruby simulator [1].

4.2. SNOOP-BASED FESI PROTOCOL RESULTS

The cache coherence simulator gives the following statistics as output:

- Read Hits and Read Misses.

- Write Hits and Write Misses.
- Writebacks: Number of times we need to write data back to LLC.
- Invalidations: Number of times invalidation requests are sent to the sharers.
- SiblingProvided: Number of times block was supplied from cache-to-cache transfers.
- fromLLC: Number of times block was supplied from LLC.
- Bus Transactions: Total number of bus transactions, BsRd + BsRdX + Invalidations.

The goals of the FESI protocol is to have less write-backs and more "SiblingProvided" transfers than the most popular currently used protocols, MESIF and MOESI. The number of invalidations would be more or less the same among all the protocols.

We simulated numerous memory traces [6] on the cache coherence simulator. We elaborate on three such memory traces that provide us insights on the performance of the protocols.

Table 4.1. Results of memory trace 1, snoop-based FESI simulation

Trace 1	MESIF	MOESI	FESI
Reads	15000	15000	15000
Read Misses	15000	15000	15000
Writes	1000	1000	1000
Write Misses	1	1	1
Writebacks	1000	0	0
Provided	15000	15000	15000
fromLLC	1	1	1
Bus Transactions	31000	31000	31000

Trace 1 is a memory trace with intermixed writes followed by reads to the same data block from other cores. This causes the 100% miss rate for reads. From trace 1, we can see the following:

- Due to dirty sharing of data blocks in MOESI and FESI, the number of write-backs is much less compared to MESIF.

Table 4.2. Results of memory trace 2, snoop-based FESI simulation

Trace 2	MESIF	MOESI	FESI
Reads	16000	16000	16000
Read Misses	16000	16000	16000
Writes	16000	16000	16000

Write Misses	0	0	0
Writebacks	15808	15808	15808
Provided	0	0	0
fromLLC	16000	16000	16000
Bus Transactions	31808	31808	47808

Trace 2 is a heavily single core memory trace. This means one core does it's work, then moves on to the second core, and so on. This is a memory trace that does not utilize the "parallel" nature of multi-core architecture. From trace 2, we can see the following:

- Due to one core doing it's work, and then only moving on to the next core, there are no sharers during eviction of 'F' block. FESI thus cannot pass on the 'F' state during eviction. This is why the number of write-backs are the same for all three protocols. However, the claim that FESI always has number of write-backs less than or equal to all other existing protocols still holds.
- Due to heavy single-core memory trace, repeated writing to the same 'F' block makes FESI send BsRdX to all cores even though there are no other sharers. In the case of MESIF and MOESI, repeated writing causes the block to go to 'M' state after which BsRdX requests need not be sent to the other cores. Trace 2 highlights the drawback in FESI protocol which arises due to the lack of 'M' state.

Table 4.3. Results of memory trace 3, snoop-based FESI simulation

Trace 3	MESIF	MOESI	FESI
Reads	16000	16000	16000
Read Misses	16000	16000	16000
Writes	16000	16000	16000
Write Misses	0	0	0
Writebacks	15999	15999	0
Provided	15999	14999	15999
fromLLC	1	1001	1
Bus Transactions	47998	63997	47998

Trace 3 is a heavily multi-core memory trace. It accesses most of the cores simultaneously, and makes use of the "parallel" nature of multi-core architectures. Due to this, there is always some sharer when a block in 'F' state gets evicted. From trace 3, we can see the following:

- On eviction of ‘O’, or during transition from ‘O’ to ‘M’, write-back happens in MOESI. On the other hand, FESI passes on ‘F’ state to sharers during eviction, thus eliminating need for write-back.
- On every transition from ‘M’ to ‘S’, MESIF requires a write-back since dirty sharing of the block is not allowed. FESI allows dirty sharing, thereby reducing the number of write-backs.
- Since ‘O’ state can only be reached from ‘M’, it cannot do “clean sharing” of data blocks. This is why MESIF and FESI have less number of “fromLLC” accesses than MOESI.
- This trace provides the perfect example of FESI achieving significantly less number of write-backs compared to MOESI and MESIF.

We ran multiple traces in addition to the three traces elaborated above. All the traces were run for MSI, MESI, MESIF, MOESI and FESI. A comparison of the total number of write-backs across all traces is given in **Fig. 4.1.**

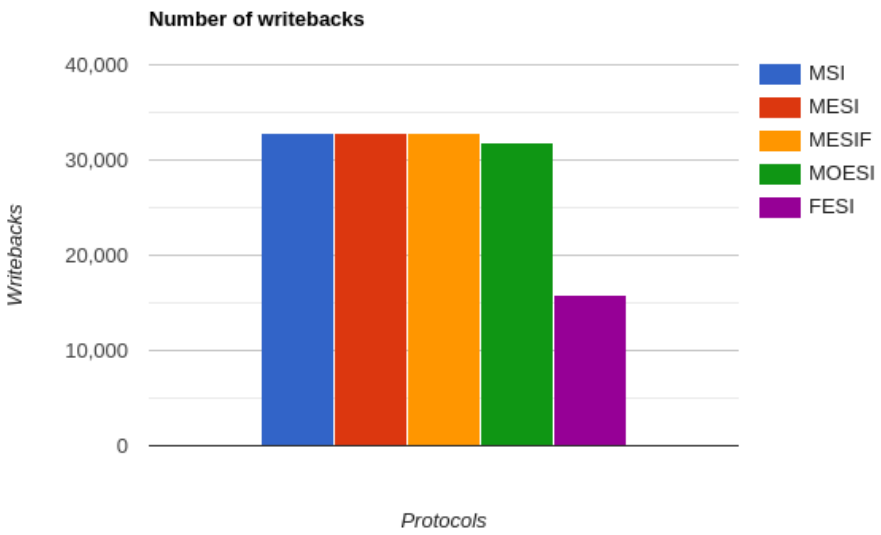


Fig. 4.1 Bar graph showing comparison of number of writebacks across MSI, MESI, MESIF, MOESI and FESI

We can see MOESI has less number of write-backs compared to MSI, MESI and MESIF. This is what was intended during the design of MOESI. But the number of write-backs in the novel FESI protocol is significantly lesser (less than half) than even MOESI.

From above, we can see that FESI achieves the least number of write-backs compared to other existing protocols. Further, FESI achieves 100% efficiency in utilization of cache-to-cache transfers over supplying from LLC. The two combined result in the least number of main memory/LLC accesses compared to any existing protocol.

From the above results, we also see that this comes at the cost of additional bus transactions. The additional bus transactions are caused by two factors:

- Need for sending BsRdX even when there are no sharers, due to lack of 'M' state.
- Additional bus transactions needed to pass on 'F' to another core during eviction.

Since bus transactions are faster than LLC/main memory accesses in a high majority of modern-day systems, FESI will give better performance than MOESI and MESIF.

4.3. DIRECTORY-BASED FESI PROTOCOL EXPERIMENTAL SETUP

In the previous section, we saw the benefits of the FESI protocol while simulating all the protocols in the locally built cache coherence simulator. After positive preliminary results from the previous section, to obtain more detailed results and performance statistics, we build the FESI protocol using the GEM5 simulator [1]. In the next sections, we present the results obtained from simulating the directory-based FESI protocol on GEM5.

4.3.1 OVERVIEW OF GEM5

GEM5 is one of the most commonly used open source system-level and processor simulators. It is very commonly used in computer architecture research. It helps in building new models on top of existing models, and making changes to individual modules in the system without affecting the rest of the system. In this project, we focus on the memory and cache part of GEM5 even though it can provide more flexible simulation environments. GEM5 consists of two memory models, 'Classic' and 'Ruby'. The default cache coherence protocol used by the classic model is MOESI, while Ruby contains multiple cache coherence protocols including MSI, MESI_TwoLevel, MESI_ThreeLevel, MOESI_hammer, etc.. Ruby is highly flexible and can model different types of coherence systems. The Ruby model is a drop-in replacement for the classic memory model on GEM5, while the rest of the system remains unaffected.

The Ruby system is preferred for modelling new cache coherence protocols because it offers an intuitive methodology to do so. While designing new coherence protocols in Ruby, the most important structure is the controller, or state machine of the coherence protocol. Ruby uses a domain-specific language called "SLICC" to implement these state machines. Each such "SLICC" state machine file consists of states, events, the transitions between states triggered by the events, and the actions taken on each such transition. Each coherence protocol is made up of multiple such SLICC state machine files, one for the cache controller, one for the directory controller, and so on. Once we design our novel cache coherence protocol by writing these SLICC state machine files, the SLICC compiler outputs a set of C++ files which can then be run with all of GEM5's other files.

For example, the SLICC state machine file for the cache controller of MSI protocol would consist of:

- States: M, S, I and other transient states.
- Events: Input messages which trigger transitions. Ex. GetS, GetM, etc.
- Transitions: Transition from one state to another on an event. Ex. 'M' to 'I' on a fwdGetM request. Each transition consists of one or more actions to be done.
- Actions: Implement what needs to be done during a transition. Ex. sendGetS, sendGetM, popRequestQueue, etc.

```
transition(M, FwdGetM, I) {
    sendCacheDataToReq;
    deallocateCacheBlock;
    popForwardQueue;
}
```

Fig. 4.2 Code showing example of Transitions in GEM5 Ruby SLICC files

Fig. 4.2 shows an example of a transition, when a core has the block in 'M' state and receives a FwdGetM request from the directory. The following actions are done during this transition: send the data block to the requestor, deallocate (invalidate) the cache block and move to I.

```
action(sendGetM, "gM", desc="Send GetM to the directory") {
    enqueue(request_out, RequestMsg, 1) {
        out_msg.addr := address;
        out_msg.Type := CoherenceRequestType:GetM;
        out_msg.Destination.add(mapAddressToMachine(address,
                                MachineType:Directory));
        out_msg.MessageSize := MessageSizeType:Control;
        out_msg.Requestor := machineID;
    }
}
```

Fig. 4.3 Code showing example of Actions in GEM5 Ruby SLICC files

Fig. 4.3 shows an example of an action, "sendGetM". For this action, we send a message on the 'request_out' queue with 'CoherenceRequestType' as GetM and destination as directory. We also send the core's "MachineID" so that the directory knows which core to respond to with the data block.

4.3.2 FESI IN GEM5

We build the directory-based FESI protocol using the Ruby system. It is a 1-level cache system. It contains three SLICC state machine files, one for directory controller, one for cache controller, and one for the message types that can be sent. The directory and cache controller SLICC files exactly follow the directory-based FESI specifications given in **Section 3.5 of this thesis**. The columns are the events, the rows are the states, and the cell contents of the table describe the transitions and actions. The 'message types' SLICC file describes the classes of two types of messages that can be sent on the network, RequestMsg for "requests" and "forwarded requests", and ResponseMsg for "responses".

The SLICC state machine files for the FESI protocol can be found in this link: https://github.com/rishabh-c-s/FESI_protocol/tree/master/FESI_GEM5

We use the IITM Aqua cluster [5] to build GEM5 with the new FESI protocol.

We test the performance using the ruby_random_tester, a tool provided by Ruby to measure the performance of cache coherence protocols. The number of load/store operations and the number of cores in the system can be specified as input to the ruby_random_tester. On executing this ruby_random_tester using the GEM5 built with FESI protocol, it builds a system with the specified number of cores and runs the specified number of load/store operations with FESI protocol. Detailed performance statistics are obtained as output from the ruby_random_tester.

Even though Ruby contains MSI, MESI and MOESI protocols, only MSI is a 1-level cache system. The MESI and MOESI protocols in Ruby are either 2-level caches or more, and provide advanced features such as DMA. Thus, in this thesis, we compare the FESI implementation with only the MSI 1-level protocol.

4.4 DIRECTORY-BASED FESI PROTOCOL RESULTS

We ran the ruby_random_tester with 10,00,000 loads/stores for both MSI 1-level and FESI 1-level protocols. We ran two iterations, first with a 4 core system and the second with a 12 core system.

Table 4.4. Results of GEM5 simulation of Directory-based FESI protocol with 4 cores

4 core system	MSI	FESI
Sim Ticks	50374501	50222191
Sim Seconds	0.050375	0.050222
Host Seconds	58.41	52.75
Read from LLC / Data from Dir	738209	9758
Data from Sibling	3162628	3722365
Write to LLC	693675	9647
PutF write-back	-	9647
PutF pass on	-	1814

Table 4.5. Results of GEM5 simulation of Directory-based FESI protocol with 12 cores

12 core system	MSI	FESI
Sim Ticks	50456041	50236651
Sim Seconds	0.050456	0.050237
Host Seconds	82.99	80.48
Read from LLC / Data from Dir	908258	105
Data from Sibling	3734528	4575730
Write to LLC	822673	3
PutF write-back	-	3
PutF pass on	-	1

From the above results, we can observe the following:

- The amount of data supplied from the LLC is substantially less in FESI compared to MSI. It further sharply decreases in FESI with the increase in number of cores.
- The amount of write-backs to LLC is substantially less in FESI compared to MSI. It further sharply decreases in FESI with the increase in number of cores.
- FESI utilizes cache-to-cache transfer more than MSI.
- More the number of cores, the stronger FESI becomes compared to other existing protocols since there are more sharers to pass on the 'F' block to on eviction.
- During ~20% of the total number of evictions of 'F', FESI eliminates the write-back by passing on 'F' to another sharer. This amounts to reducing the number of write-backs by **20% compared to MOESI and MESIF**.

CHAPTER 5

POSSIBLE ENHANCEMENTS TO FESI

5.1. SENDING 'PUTF' WITHOUT DATA

Currently in FESI, the PutF request is always sent with the data. If the directory can find a sharer to pass on the 'F' state, this data is not used. Otherwise, the data is written back to the LLC. This can be seen from the state transition diagram in **Fig. 5.1.**

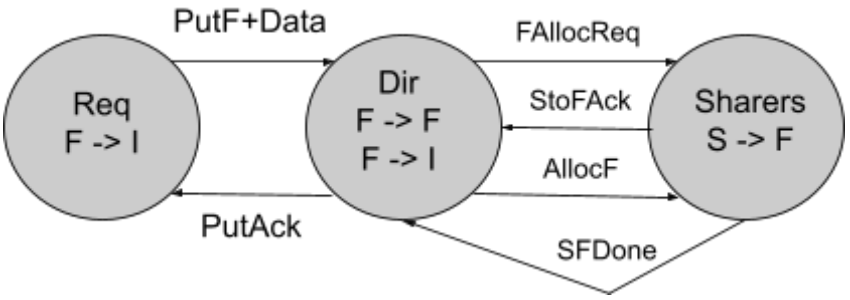


Fig. 5.1 State transition diagram showing PutF+Data transition in FESI protocol

From simulating the FESI protocol on GEM5, we obtain the following statistics regarding the number of times 'F' gets passed to another core on eviction vs number of times write-back is required on eviction:

Table 5.1. GEM5 statistics to decide on sending PutF with data vs without data

	PutF write-back	PutF pass on
4 cores	9647	1814
12 cores	3	1
20 cores	0	2

For an even comparison, the 12 cores system is tested with 10,00,000 loads/stores while the 20 cores system is tested with 20,00,000 loads/stores. Apart from the observation that the probability of 'F' getting evicted decreases with increasing number of cores, we can also see the following:

- Out of the total number of PutF, 'F' gets passed on to another sharer ~20% of the time.
- With an increasing number of cores, this percentage increases. Ex. 100% for 20 cores.

When ‘F’ gets passed on to another sharer, the data block is not needed since write-back is not required. Thus, sending data along with the PutF request is a waste. Cache data blocks are usually 64B in size. Sending 64B of data with each PutF causes network congestion which in turn might affect performance. Thus, an alternate implementation of FESI could be sending PutF to the directory without data. If the directory is not able to find a sharer to pass on ‘F’, it will request for the data block, following which the core sends the data.

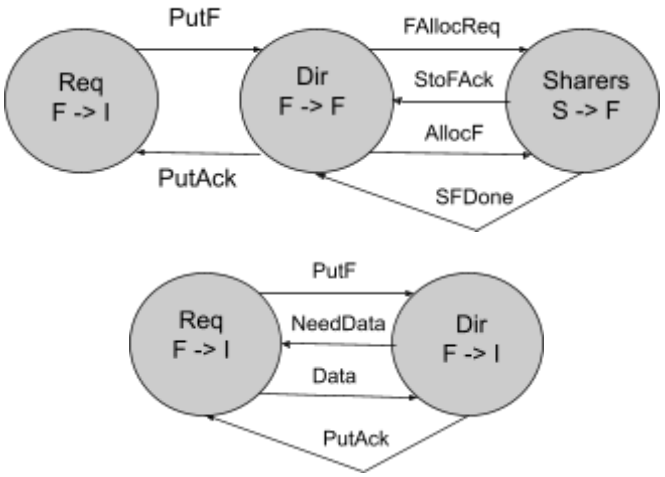


Fig. 5.2 State transition diagram showing ‘PutF without data’ enhancement to FESI protocol

In the top state transition diagram in **Fig. 5.2,** there is a sharer with the data block. Thus, the directory passes on ‘F’ state to the sharer and sends PutAck to the requestor. In the second state transition diagram in **Fig. 5.2**, there is no other sharer to pass on the ‘F’ state. Thus, the directory sends ‘NeedData’ to the requestor, and writes back the data to the LLC.

Since 20% of the time during ‘F’ eviction, a sharer is available, the alternate implementation saves needless network congestion. However, in case a sharer is not available, extra requests and responses need to be done, which might cause a delay. A trade-off is present between time lost in sending data always vs time lost in extra “NeedData” requests when no sharer is available. This trade-off is not explored in this thesis, and is scope for future work.

5.2. ADDING THE ‘M’ STATE

The only limitation to the FESI protocol is that on repeated ProcWrite requests to the same data block by the same core, even though the first write request would invalidate all other sharers, we still need to send unnecessary BsRdX/GetM requests for the subsequent writes. This is due to the lack of the ‘M’ state.

On analyzing the simulation results from running FESI on GEM5, the following results are obtained:

Table 5.2. GEM5 statistics to show advantage in adding 'M' state

	ProcWr to F with Sharers	ProcWr to F without Sharers
4 cores	170019	818219
12 cores	67672	261711
20 cores	84729	311108

We can see that ~80% of the time that a 'F' block receives a ProcWrite request, it has no sharers. Thus, ~80% of the time, we are wasting a network transaction where the requestor sends GetM and the directory responds with AckCount = 0. This can be solved by adding a conventional 'M' state, where the block moves to the 'M' state on a ProcWrite. The 'M' state will have Read/Write privileges.

This new protocol can be called the "MESIP" protocol to avoid confusion with the existing "MESIF" protocol. The 'P' state is the 'Provider' state which is essentially the same as the 'F' state of FESI protocol. The MESIP protocol would also achieve the same goals as the FESI with 100% utilization of cache-to-cache transitions over LLC accesses and minimal write-backs. Additionally, MESIP would also solve the issue of repeated writes to the same block from the same core.

The trade-off between FESI and MESIP:

- FESI is a 4-state protocol, thus needs 2 bits per cache block to represent the state. MESIP is a 5-state protocol, thus needs 3 bits per cache block. FESI consumes less memory.
- MESIP saves on unnecessary GetM transitions, thereby marginally improving performance.

This trade-off is not explored in this thesis, and is scope for future work.

CHAPTER 6
CONCLUSION & FUTURE WORK

6.1. CONCLUSION

Cache-to-cache supply of data blocks (sibling transfer) is faster than supplying from the LLC. Thus, 100% utilization of cache-to-cache transfer over supplying from the LLC would improve the performance of the cache coherence protocol. We noticed that the existing protocols, especially the two most popular protocols today, MESIF and MOESI do not achieve this 100% utilization of cache-to-cache transfers. In Chapter 2, we have elaborated scenarios where both MESIF and MOESI would supply a block from the LLC even when a sibling core holds the data block. This subsequently also causes the problem of unnecessary write-backs to the LLC. Overall, the LLC is accessed more than it needs to be in the existing protocols.

We proposed "FESI", a novel 4-state invalidation-based coherence protocol. We achieved the lowest possible LLC accesses through this protocol. This was done by combining the best of MESIF and MOESI. FESI rotates the 'F' block among sharers on a MRU basis similar to the MESIF protocol. This way, there is no huge burden on a single core to keep supplying the block. FESI allows dirty cache-to-cache transfers of blocks similar to MOESI protocol. This eliminated the need for write-back on the 'M' to 'S' transition. Additionally, we proposed the method of "passing on the F state" to other sharers on eviction. This way, on eviction of a 'F' block, there would still be another core in 'F' state to supply the data block, and thus FESI does not need to supply from the LLC. Also, passing on the 'F' state allows FESI to defer the write-back. Thus, FESI achieves both 100% utilization of cache-to-cache transfers over reading from LLC and minimal write-backs to the LLC, automatically resulting in the least number of overall LLC accesses possible among all existing coherence protocols.

6.2. FUTURE WORK

The FESI protocol achieves the least number of LLC accesses compared to all existing coherence protocols at the cost of additional bus transactions. We need to explore methods to reduce the number of bus transactions and network congestion in future work. We have provided two such possible methods in Chapter 5 of this thesis: not sending data along with PutF and adding the 'M' state to the protocol (MESIP protocol).

Besides, there are few hurdles in the GEM5 simulator [1] which prevent us from conducting a fair comparison of the different cache coherence protocols. The Ruby memory model does not consist of the MESIF protocol. Additionally, even though Ruby has MSI, MESI and MOESI protocols, all these are with different cache hierarchies. MSI is a 1-level cache hierarchy, while MESI is a 2-level cache hierarchy. Also, some protocols have DMA while other protocols do not. To enable a fair comparison among all protocols, MSI, MESI, MESIF, MOESI and FESI, we need to build 1-level cache hierarchy implementations of MESI, MESIF and MOESI in the GEM5 Ruby simulator [1].

REFERENCES

[1] The GEM5 Simulator System, available at http://www.gem5.org , Feb 2013.

[2] Learning GEM5 website and book, by Prof. Jason Lowe-Power, available at www.gem5.org/documentation/learning_gem5/introduction/

[3] A Primer on Memory Consistency and Cache Coherence, book by Daniel J. Sorin, Mark D. Hill, David A. Wood

[4] Course Slides of CS6600:Computer Architecture course at IITM, by Madhu Mutyam

[5] AQUA Cluster IITM and how to use it, available at https://hpce.iitm.ac.in/

[6] Large Memory Traces for Cache Coherence Simulators, from Kshitizdange's Github project available at https://github.com/kshitizdange/418CacheSim/tree/master/traces

APPENDIX

Installing GEM5 from Source Files in RHEL7.6

Before proceeding to build GEM5, we need to install the following:

- Libffi-devel
- Openssl
- Python3.6+
- SCons & required packages

Many times, we are in a situation where we do not have root access to a server. The major challenge then is we need to install all of the above through source files. This appendix gives a step-by-step walkthrough on how to install GEM5 and all its dependencies from source files.

NOTE: Hereafter in this document, we call your working directory (where you want to install gem5 for your project) as $HOME.

Using the screen command

When you connect to another server (like the Aqua cluster [5]) using 'ssh', you might get logged out due to internet issues or server timeout settings. In these cases, if you get logged out, the command currently running, and the outputs displayed get deleted when you log back in. Thus, when you are running time-intensive commands, such as "make", there is a high risk of getting logged out and losing your progress.

For this purpose, we use the "screen" command. Just type the command "screen" and press enter. You will enter a screen. This is just like another terminal, and can run your commands as usual. In case you get logged out of the server, log back in, type "screen -d -r" and you will reconnect to the previous screen, and the process will still be running.

If you want to run a process in screen, and then come back to your regular server terminal, just use the keyboard shortcut; "ctrl + a" and then "d". You will be "detached" from your screen. If you want to reconnect, type "screen -d -r" as mentioned before.

Through this, you can ensure you are not at the risk of losing your progress. We suggest you run all the "configure", "make", "make install" and "build" commands through a screen.

Installing Libffi-devel

- Go to the $HOME directory
- "wget https://centos.pkgs.org/7/centos-x86_64/libffi-devel-3.0.13-19.el7.x86_64.rpm.html"
- "rpm2cpio ./libffi-devel-3.0.13-18.el7.x86_64.rpm | cpio -idmv"
- A "usr" directory would be automatically formed inside the $HOME directory
- Go to $HOME/usr/share.
- Create a file "config.site" and add the following two lines to the file:
    - CPPFLAGS=-I$HOME/usr/include
    - LDFLAGS=-L$HOME/usr/lib64

Installing Openssl

- Cd $HOME/usr
- wget https://www.openssl.org/source/openssl-1.0.2o.tar.gz
- Tar xvf https://www.openssl.org/source/openssl-1.0.2o.tar.gz
- cd openssl-1.0.2o
- ./config shared --prefix=$HOME/usr
- Make
- Make install

Setting the Library, Package config and Binary paths

The following lines need to be added to the ~/.bash_profile file.

- export PATH=$HOME/usr/bin:$PATH
- export PATH_SCREEN=$PATH
- export LDFLAGS="-rdynamic"
- export LDFLAGS_SCREEN=$LDFLAGS
- export SETUPTOOLS_USE_DISTUTILS=stdlib
- export SETUPTOOLS_USE_DISTUTILS_SCREEN=$SETUPTOOLS_USE_DISTUTILS
- export LD_LIBRARY_PATH=$HOME/usr/lib64$HOME/usr/lib:$LD_LIBRARY_PATH
- export PKG_CONFIG_PATH=$HOME/usr/lib64/pkgconfig:$PKG_CONFIG_PATH

- export LD_LIBRARY_PATH_SCREEN=$LD_LIBRARY_PATH
- export PKG_CONFIG_PATH_SCREEN=$PKG_CONFIG_PATH

Now add the following lines to the ~/.screenrc file (if not present, create one). This is to ensure the environment variables are also passed to whichever screen you might be using.

- setenv LD_LIBRARY_PATH $LD_LIBRARY_PATH_SCREEN
- setenv PATH $PATH_SCREEN
- setenv SETUPTOOLS_USE_DISTUTILS $SETUPTOOLS_USE_DISTUTILS_SCREEN
- setenv LDFLAGS $LDFLAGS_SCREEN
- setenv PKG_CONFIG_PATH $PKG_CONFIG_PATH_SCREEN

After adding the above lines, if it is your own machine, restart the machine. If you are connecting to some other server through ssh, logout and log back in.

Check if the above environment variables have been updated successfully by doing "echo $(name)" and checking the output. For example, echo $LDFLAGS should print -rdynamic. Check the same in the Screen too by running "echo" after entering a screen. If it has not been updated, terminate the screen, logout from the server, login again and check. If it is still not working, check if you have entered the values in the .bash_profile and .screenrc files correctly.

Installing Python3.7

GEM5 needs Python3.6+. The following instructions are for installing Python3.7, but will also work for the later versions.

- Come to the $HOME directory.
- wget https://www.python.org/ftp/python/3.7.0/Python-3.7.0.tgz
- Tar xvf https://www.python.org/ftp/python/3.7.0/Python-3.7.0.tgz
- ./configure --prefix=$HOME/usr/ --enable-shared --enable-optimizations
- Make
- Make altinstall

The Python3.7 binary would now be installed in the $HOME/usr/bin folder. However, GEM5 [1] will search for the names “Python” and “Python3”. Thus, go to $HOME/usr/bin and create the following symbolic links:

- python → python3.7
- python3 → python3.7
- python-config → python3.7m-config
- python3-config → python3.7m-config

You can check whether it is indeed the latest version of Python, and whether the links are working properly by the following methods:

- The commands “which python”, “which python3” and “which python3.7” should give the same output, ie, $HOME/usr/bin
- Run all “python --version”, “python3 --version” and “python3.7 --version” and ensure it is that which you installed.

Similarly, we need to create the links for the Python libraries too. Go to $HOME/usr/lib/ folder, and you will find libpython3.7m.a and libpython3.7m.so.1.0 files. Create the following symbolic links in the $HOME/usr/lib folder:

- libpython3.7m.so -> libpython3.7m.so.1.0
- libpython.a -> libpython3.7m.a
- libpython.so.1.0 -> libpython3.7m.so.1.0

That concludes the installation of Python3.7. For one final installation verification, do the following:

- Run the command “ldd $HOME/usr/bin/python”
- Ensure there is the following line in the output; “libpython3.7m.so.1.0 => $HOME/usr/lib/libpython3.7m.so.1.0”

Getting PIP

Pip is a very powerful Python tool which will help us install subsequent required packages.

- Cd ~
- curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py
- python3 get-pip.py

Installing Protoc

- Cd $HOME
- wget https://github.com/protocolbuffers/protobuf/releases/download/v3.15.8/protoc-3.15.8-linux-x86_64.zip
- Unzip protoc-3.15.8-linux-x86_64.zip
- After running the unzip command, two folders, “bin” and “include” would have been created in the $HOME directory.
- Move the files in this $HOME/bin directory to $HOME/usr/bin and the files in this $HOME/include directory to $HOME/usr/include.
- You can now check installation of protoc by running the “protoc --version” command. You should get the output as “libprotoc 3.15.8”.

Installing SCons & other packages through Pip

We can install packages through pip using the command “pip install <package_name>” and uninstall using the command “pip uninstall <package_name>”. We can check the list of all packages currently installed in the system using “pip list”. We need all the packages given in **Fig. A.1.**

```
[ee17b067@aqua sample_my_btp]$ pip list
Package            Version
---------------- -----------
bleach             2.1
colorama           0.3.3
grpcio             1.18.0
grpcio-tools       1.18.0
html5lib           1.1
packaging          20.9
pip                21.0.1
protobuf           3.15.8
protobuf-compiler 1.0.20
pybind11           2.6.2
pyparsing          2.4.7
SCons              4.1.0.post1
setuptools         56.0.0
six                1.15.0
termcolor          1.1.0
tqdm               4.31.1
webencodings       0.5.1
[ee17b067@aqua sample_my_btp]$
```

Fig. A.1 Image showing list of pip packages required for GEM5

If you run “pip list” in your terminal, most of the packages shown in **Fig. A.1** would be already installed. However, they would have been installed by the System Python, and not the Python3.7 that we installed. Thus, we need to uninstall all the packages, and install again.

- Run “pip list”
- Uninstall every package except pip using “pip uninstall <package_name>”.
- Install every package given in **Fig. A.1**, except “SCons” using “pip install <package_name>”
- Only once every package (except SCons) has been installed can we install SCons. Run “pip install scons”. NOTE: “scons” is in all small letters unlike the actual package name.

Building GEM5

Now that we have installed all the required dependencies, we can proceed to build gem5:

- cd $HOME/usr
- git clone https://gem5.googlesource.com/public/gem5
- Cd $HOME/usr/gem5
- scons build/X86/gem5.opt -j 16
    - 16 denotes the number of cores gem5 will build with, can be modified
    - This step takes around 20-40 minutes to run

GEM5 has now been successfully installed in your system! You can learn to use GEM5 through this link: https://www.gem5.org/documentation/learning_gem5/gem5_101/